Integração do sistema Luveo com o Claude

Especificação para o desenvolvedor — servidor MCP (Model Context Protocol)
Sistema: https://agt.luveo.com.br/operacional

1. Objetivo

Permitir que o Claude (claude.ai, app desktop e Cowork) consulte diretamente o banco de dados e os relatórios do sistema Luveo, para responder perguntas como *"quais processos chegam esta semana?"* ou *"gere o relatório operacional de setembro"*.

A forma padrão de fazer isso é criar um **servidor MCP** (Model Context Protocol), protocolo aberto que o Claude usa para se conectar a sistemas externos. O servidor MCP expõe **ferramentas** (funções) que o Claude pode chamar. Depois de publicado, ele é adicionado no Claude como **conector personalizado** (custom connector).

2. Arquitetura resumida

- O backend do Luveo parece ser em **FastAPI** (Python) — confirmar.
- Adicionar um endpoint MCP, por exemplo `https://agt.luveo.com.br/mcp`, usando a biblioteca **FastMCP**.
- Transporte: HTTP (Streamable HTTP), obrigatoriamente com HTTPS.
- Autenticação: OAuth (recomendado) — os dados são da empresa, o endpoint não pode ficar aberto.
- O usuário adiciona a URL no Claude: Configurações > Conectores > Adicionar conector personalizado.

3. Instalação

```
pip install fastmcp
```

4. Opção A — converter a API existente automaticamente (mais rápido)

O FastMCP consegue transformar as rotas FastAPI existentes em ferramentas MCP:

```
from fastmcp import FastMCP
from main import app  # app FastAPI existente do Luveo

# Converte as rotas FastAPI existentes em ferramentas que o Claude pode chamar
mcp = FastMCP.from_fastapi(app=app, name="Luveo")

if __name__ == "__main__":
    mcp.run(transport="http", host="0.0.0.0", port=8001)
```

Atenção: isso expõe todas as rotas, inclusive as que alteram ou apagam dados. Filtrar para expor somente rotas de leitura (GET), ou preferir a Opção B.

5. Opção B — ferramentas escritas manualmente (recomendado)

Mais seguro e dá respostas melhores, porque cada ferramenta tem uma descrição clara. O Claude escolhe qual ferramenta usar com base na *docstring* — pode ser escrita em português.

```
from fastmcp import FastMCP

mcp = FastMCP("Luveo")

@mcp.tool
def listar_processos(status: str | None = None) -> list[dict]:
    """Lista processos de importação, com filtro opcional por status
    (ex.: 'em trânsito', 'desembaraçado', 'entregue')."""
    return db.query_processos(status)        # código de consulta existente

@mcp.tool
def relatorio_operacional(data_inicio: str, data_fim: str) -> dict:
    """Relatório operacional entre duas datas (formato YYYY-MM-DD)."""
    return gerar_relatorio(data_inicio, data_fim)

@mcp.tool
def detalhe_processo(referencia: str) -> dict:
    """Retorna todos os dados de um processo pela referência interna."""
    return db.get_processo(referencia)
```

Sugestão de primeiras ferramentas (somente leitura): listar processos / embarques com filtros; detalhe de um processo; chegadas previstas (ETA) por período; relatórios operacionais existentes; busca por cliente, fornecedor ou NCM.

6. Montar dentro do app FastAPI existente (mesmo servidor)

```
from fastapi import FastAPI
from mcp_server import mcp   # instância FastMCP da Opção B

mcp_app = mcp.http_app(path="/")
app = FastAPI(lifespan=mcp_app.lifespan)   # o lifespan é obrigatório
app.mount("/mcp", mcp_app)
# ... rotas existentes do Luveo continuam normalmente
```

Obs.: se o app já tiver um *lifespan* próprio, é preciso combinar os dois. Conferir a documentação do FastMCP para a versão instalada.

7. Segurança — requisitos

- HTTPS obrigatório.
- Autenticação OAuth no endpoint MCP (o FastMCP tem helpers de autenticação). O conector personalizado do Claude suporta login OAuth.
- Começar **somente com leitura**: nenhuma ferramenta que edite, apague ou envie dados.
- Usar um usuário de banco de dados com permissão só de leitura para o MCP.
- Limitar o volume retornado (paginação / limite de linhas) e registrar log das chamadas.

8. Testes antes de conectar ao Claude

- Rodar o servidor localmente e testar com o **MCP Inspector**: `npx @modelcontextprotocol/inspector`
- Verificar se todas as ferramentas aparecem, com descrições claras, e se retornam JSON correto.
- Publicar em produção com HTTPS e enviar a URL final (ex.: https://agt.luveo.com.br/mcp).

9. Conectar no Claude

- No Claude: Configurações > Conectores > Adicionar conector personalizado.
- Colar a URL do servidor MCP e fazer o login (OAuth).
- O conector fica disponível nos chats, no app desktop e no Cowork.

10. Entregáveis esperados do desenvolvedor

- Servidor MCP publicado com HTTPS e autenticação.
- Lista das ferramentas expostas, com descrição de cada uma.
- Usuário de teste para validar a conexão no Claude.

Referências: documentação do FastMCP (gofastmcp.com) e do Model Context Protocol (modelcontextprotocol.io).